Playwright 测试隐性误修复研究

Research on silent false healing in Playwright test repair

高东彪

（软视软件（苏州）有限公司，江苏苏州 215000）

Gao Dongbiao

(Zoom Video Communications (Suzhou), Inc., Suzhou 215000, China)

摘 要：为降低 Playwright 测试中脆弱定位器维护成本，本文研究大语言模型（Large Language Model，LLM）驱动的测试自愈是否产生隐性误修复。本文以 ReproBreak 数据集 koenig 项目 75 个真实可达失效用例为对象，在选择器解析层面测试 qwen2.5-coder:7b，并构建基于抽象语法树（Abstract Syntax Tree，AST）的 HealReact 静态基底。结果表明，目标元素被移除时，78.7% 的用例生成指向错误元素的可解析选择器，静态基底覆盖 80.6% 的真实失效目标。本文认为，无人值守持续集成（Continuous Integration，CI）自愈需要行为重放验证器。

关键词：测试自愈；大语言模型；Playwright；React；隐性误修复

中图分类号：TP311.5　**文献标识码：**A

Abstract: To reduce the maintenance cost of brittle locators in Playwright tests, this paper studies whether large language model (LLM)-driven test self-healing causes silent false healing. Using 75 real reachable break cases from the koenig project in the ReproBreak dataset, qwen2.5-coder:7b is evaluated at the selector-resolution level, and an abstract syntax tree-based HealReact static substrate is built. The results show that, when the target element is removed, 78.7% of cases produce resolver-valid selectors pointing to wrong elements, and the static substrate covers 80.6% of real break targets. The study concludes that unattended continuous integration self-healing requires a behavioral replay verifier.

Key words: self-healing test; large language model; Playwright; React; silent false healing

0 引言

维护脆弱的用户界面（User Interface，UI）测试套件是现代 Web 工程中持续存在的维护成本。一次层叠样式表（Cascading Style Sheets，CSS）类名重命名、一次 React 组件的重新嵌套，或是一次行为不变的处理函数重构，都足以让数十个端到端（End-to-End，E2E）Playwright 测试失效。这一问题在图形用户界面（Graphical User Interface，GUI）测试录制重放与自动生成的研究综述中被多次列为关键挑战 [1-2]。国内行业期刊也持续关注软件测试自动化方法和测试技术应用形势，相关研究为工程化测试维护问题提供了应用背景 [3-4]。无论是工业界工具（如 Healenium [5]）还是日益增长的学术工作 [6-7]，相关方案给出的自然回应都是自愈：在定位器失效时，修复器自动将其重写为指向新文档对象模型（Document Object Model，DOM）中等价元素的形式。近年来，大语言模型（Large Language Model，LLM）被广泛引入软件测试与修复任务 [8-9]，自动测试修复（含定位器修复）是其中最直接受益的方向之一。

这一前景看似乐观，近期文献也使该问题看起来接近解决。Joseph [6] 通过一个确定性可访问性阶梯在单一演示样例上报告了 100% 通过率；当代基于 LLM 的定位器修复工作通常借助一致性安全约束报告修复成功率或解释一致性。然而这些指标共享一处结构性盲区：它们衡量的是修复后的测试是否通过，而非补丁是否指向开发者真正期望的元素。一个总能找到“某个”可解析选择器（任何属性近似的元素皆可）的修复器存在隐蔽的危险性——每一次“通过但错指”的修复都会掩盖一次本应被作为回归暴露的真实 UI 变更。

文章在 ReproBreak [7] 数据集中的真实 Playwright 失效用例上直接测量这种失败模式。在 koenig 的 75 个可达失效用例上，研究结果显示出明显风险：

- **第一项发现：隐性误修复是默认行为，且软提示放弃约束无效。**当真实目标元素被强制从候选集中移除时，朴素 LLM 修复器（`qwen2.5-coder:7b` [10]）在 **78.7%** 的用例中

作者简介：高东彪（1986—），男，工程师，学士；研究方向：AI 在软件测试中的应用。

通信作者：高东彪（1986—），男，工程师，学士；研究方向：AI 在软件测试中的应用。E-mail：gdb_1986@163.com。

给出语法正确但元素错误的选择器（59/75）；仅 17.3% 的情形下能正确放弃。在系统提示中追加明确的“如无候选契合则放弃”约束后，结果数字完全一致（两版本均为 59/75，$\Delta = 0$），表明模型并未将该规则纳入决策。

- **第二项发现：两个低开销确定性手段带来零回归的局部改进。**HealReact 的 L1 静态基底在 koenig 上可达 80.6%（75/93）的真实失效目标；在可达用例上，L3 修复器首位候选选择器的精确匹配率为 77.3%（58/75）。进一步地，testId 加权检索与强锚点后置过滤经二因子析因分离后各带来边际改进、合计 +3 个新正确用例且零回归（方向一致，但在 $n = 75$ 下 McNemar 未达显著）。
- **第三项发现：跨应用泛化受锚点约定差异的制约。**在另一 React+Playwright 代码库（`payloadcms/payload`，319 个失效配对）上，由于其采用 BEM 风格的模板字面量类名、当前静态抽取器无法对其求值，原始可达率从 koenig 的 80.6% 跌至 3.8%。本文将其诊断为一处明确且可修复的盲区，而非对方法本身的否证。

前两个发现为无人值守自愈流水线引入行为重放验证层提供了直接的经验性动机。第三个发现给出一条可证伪的跨应用泛化论断：任何仅在单一应用上完成基准测试的定位器修复工具，都很可能高估其泛化能力，因为锚点约定（`data-testid` vs `data-kg-*` vs BEM 模板字面量）在不同 React 代码库之间存在显著差异。

此外，文章介绍 HealReact 的 L1 基底——一个抽象语法树（Abstract Syntax Tree，AST）抽取器加一个带确定性后置校准的 LLM 派生意图标注器——该基底静态可达 koenig 93 个失效目标中的 80.6%；在其之上，一个小型本地 L3 修复器在可达用例上给出正确选择器的比例为 77.3%（在完整数据集上为 62.4% 的静态修复代理指标[1]）。两个低开销确定性杠杆——testId 加权检索与强锚点后置过滤——经二因子析因分离后各带来边际改进、合计 3 个新正确用例、零回归（方向一致，McNemar 未达显著），表明确定性后处理可作为比提示工程更可靠的安全网。

文章贡献：（i）在真实 Playwright 数据集上对隐性误修复的诊断性测量，以及对软提示放弃指令无效的展示（§3）；（ii）HealReact 的 L1 基底作为一个可证伪、可本地运行的静态层（§2）；（iii）两个确定性安全杠杆经二因子析因分离，各自带来边际改进、合计 3 增 0 减（方向一致但在 $n = 75$ 下 McNemar 未显著，见 §3）。

78.7% 是选择器层面的对抗性上界（对抗性构造）；将其与 koenig 真正不可达的 18/93 用例朴素相乘，得到无人值守工作负载下约 15% 的粗略上界估计（而非测得的部署率，详见 §3）。即便作为上界估计，文章（§4）仍认为该风险已经高到足以拒绝在持续集成（Continuous Integration，CI）中无人值守地自动提交 LLM 提出的选择器补丁。

[1]文章全文使用“静态修复代理指标”而非“端到端修复率”：该指标是选择器解析层面的替代评估指标。解析器在元素层面的精确一致性与 Playwright 通过率相关，但既不是其下界也不是其上界（静态精确匹配的选择器可能在可见性/时序约束下运行时失败；静态不匹配的选择器也可能在重渲染 DOM 中运行时通过）。两者之间的换算需要基于 Docker 的真实重放。

1 背景与相关工作

1.1 定位器脆性与基准数据集

ReproBreak [7] 是迄今最大的可复现定位器失效数据集（来自 359 个带 Cypress/Playwright 测试的 Web 仓库的 449 个可复现失效）。文章将 `tryghost/koenig` [11] 的 Playwright 切片（93 个可复现失效，分布于 19 个版本提交）用作主要基底。

1.2 自愈工具

测试自愈可视为测试失败后的自动修复问题：系统需要在既有测试语义约束下生成候选补丁，并避免“通过但错误”的补丁被接受；自动程序修复综述为这一修复框架提供了相邻背景 [12]。Healenium [5] 在运行时通过 DOM 相似度修复 Selenium 选择器，是最流行的工业界基线。Joseph [6] 基于 DOM 可访问性树确定性地重新抽取选择器，在单个电子商务演示样例上报告了 100% 通过率，是确定性静态修复轴上最强的已发表基线，也是与本文 L1 基底最直接的比较对象。据本文检索范围，尚未见有同行评议的 LLM 定位器修复工具在真实 Playwright 基准数据集上报告隐性误修复率；与之最接近的当代工作主要关注修复成功率或解释一致性，两者均不直接约束本文探针所针对的失败模式。

1.3 缺失的部分

上述工作均未在真实基准数据集上隔离隐性误修复——即修复器给出语法通过、但指向错误元素的选择器、从而遮蔽真实 UI 变更的比率。Joseph 在一个目标答案按构造一定可达的演示样例上报告了通过率；ReproBreak 提供了数据基底但未在其上评估任何修复器。§3 通过一个对抗性构造的探针填补这一空缺。

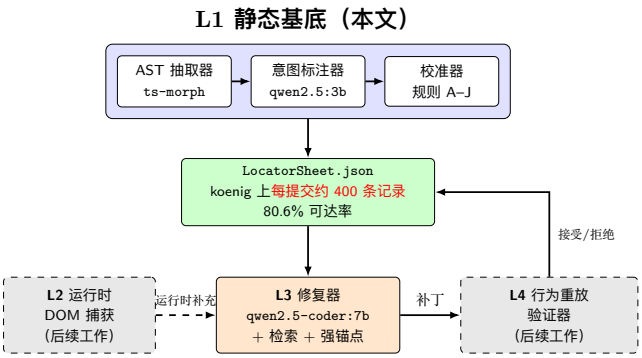


图 1　HealReact 流水线。L1（本文）是静态 AST 锚点基底，意图标注层在本文中仅在 32 元素的小规模构造样例集上评估；L3 是 §3 中评估的 LLM 修复器。L2（运行时 DOM 捕获）与 L4（行为重放验证器）以虚线框表示，二者在本文中均为设计占位、留作后续工作：L2 负责静态不可见的运行时元素，L4 的必要性由 §3 中的对抗性误修复探针在经验上得到充分动机化。

1.4 React 相关细节

`koenig-lexical` [11, 13] 是一个基于 React + Lexical 的现代富文本编辑器，其 Playwright 套件在很大程度上依赖自定义 `data-kg-*` 属性（约占其失效的一半），而非 `data-testid`。`payloadcms/payload` [14] 是一个 Next.js 后台 UI，其 CSS 类名遵循块、元素、修饰符（Block Element Modifier，BEM）方法 [15]，使用 `${baseClass}__suffix` 模板字面量——这些字面量对 React 可见，但对纯字面量的 AST 抽取器不可见。这两种锚点约定的对比是文章跨应用发现的依据（见 §3.5）。

2 HealReact L1 静态基底

HealReact 设想为一条按“静态 → 行为验证”递进的四层流水线（编号即由静态程度递减、行为验证程度递增的次序给出）：**L1**（静态 AST 锚点基底，本文）从源码静态抽取候选元素；**L2**（运行时 DOM 捕获层）补足仅在运行时才渲染、静态不可见的元素；**L3**（LLM 选择器修复器，本文）在前两层产出的候选上推断修复后的选择器；**L4**（行为重放验证器）以真实重放证伪“通过但错指”的补丁。这套 L1–L4 编号是本文对 HealReact 系统的自定义层级命名，并非引自外部框架。本文实现并评估 L1 与 L3；L2 与 L4 在本文中仅作设计占位（分别对应运行时 DOM 捕获与行为重放验证，均留作后续工作）。

HealReact 的 L1 层是 L3 修复器与未来行为验证器（L4）所依托的静态基底。它由三部分组成（图 1）。

2.1 AST 抽取器

基于 `ts-morph` [16] 构建的抽取器遍历目标目录下的每个 `.tsx`/`.jsx` 文件，并为每个交互式 JavaScript XML（JSX）元素发出一条 JavaScript 对象表示法（JavaScript Object Notation，JSON）`LocatorSheet` 记录。每条记录包含元素 tag、父链、role、`data-testid`、`aria-label`、`id`、`className`、自由文本内容，关键还包括任意自定义 `data-*` 属性（捕获在 `dataAttrs` 映射中）。抽取器还把任何承载稳定锚点（`data-testid`、`data-cy`、`data-kg-*`、`aria-label`，或驼峰 `dataTestId`/`testId`/`testID` prop）的 JSX 元素都视为可交互——而不仅是规范超文本标记语言（HyperText Markup Language，HTML）tag 集。仅这一项改动就将 `koenig-lexical` 上的锚点覆盖率从 22% 提升到 48%、解析器命中率从 0% 提升到 71%，说明现代 React 的脆性中有相当部分藏在自定义包装组件里。

2.2 意图标注器与校准器

HealReact 还包含一个 LLM 意图标注流程（经 Ollama [17] 运行的 `qwen2.5:3b`），它依据组件源码与 JSX 上下文为每个交互元素生成简短的功能性意图标签（例如 `submit-order-button`、`toggle-email-notification`）；随后由一个十条规则（A–J）构成的确定性校准层负责识别并降低可疑标签的置信度，可疑模式包括兄弟集合的塌缩标签、把 HTML tag 直接当作名词的退化、以及 role 与 intent 不匹配等。被标记的记录将被重新提交给 `qwen2.5-coder:7b` 重打标签，形成“先低开销、后高开销”的分级验证机制。**范围说明**：本文对意图层的评估限定在 32 元素的构造样例集上；§3 中针对真实失效用例的所有数字仅使用抽取器输出的锚点与词法检索，不使用 L1 意图标签。真实代码库上对意图稳定性的大规模评估超出本文范围。

2.3 定位器解析器作为可证伪的透镜

文章构建了一个 Playwright 选择器解析/匹配器，它接受任意选择器表达式（`page.locator`、`getByTestId`、`getByRole`、`waitForSelector`、原始 CSS）并返回它会匹配的 LocatorSheet 记录集。这是 §3 中各项论断的判定工具：可达性是“失效选择器是否能解析到 sheet 中的某条记录？”；L3 正确性是“解析器是否把 LLM 提议的选择器与真值选择器映射到同一元素？”。构建这个判定工具要求研究处理引号感知的嵌套参数、动态 JavaScript（JS）表达式属性值、以及裸属性选择器；早期版本由于解析器缺陷把可达率虚抬到 85–98%，文章现在在诚实性审计中明确标注了这一点。

2.4 核心指标

在 koenig-lexical 的 19 个失效版本提交上，抽取器在每个版本提交中平均产出约 400 条 LocatorSheet 记录（跨提交范围 314–489），解析器在静态层面可达真实 Playwright 失效目标的 **75 / 93 = 80.6%**。剩余 18 个用例需要运行时 DOM（例如 Lexical 框架在运行时才渲染出的 <p> 节点），属于经典的运行时 DOM 捕获范畴，由 HealReact 的 L2 层负责。

3 三个实证发现

下文所有数字都可由开放脚本 heal_baseline.py、false_heal_probe.py 与 cross_app_probe.py 在 healreact/bench/cases/ 下的 JSON 工件上复现。完整实验扫描（两个 L3 变体和两个误修复探针）在 Apple M5 Pro / 48 GB 上仅用 Apple Metal 加速时，实际运行时间约为 8 min。

3.1 复现配置

为便于复现，本文固定以下设置：L3 修复器与意图标注器分别经 Ollama 运行 qwen2.5-coder:7b 与 qwen2.5:3b，二者均取 temperature=0（贪心解码、单次采样）；候选检索采用加性词元重叠打分，权重为 testId×3、dataAttr key/value×2、aria/id/class/text/path×1，取前 k 个候选（k 与各探针配置一并记录在工件 JSON 中）。每个失效用例的逐条记录（用例编号、koenig 所属版本提交、失效选择器、ReproBreak new_locator、模型给出的选择器与所选候选下标、推理说明、选择器语法校验结果、解析得到的目标元素、是否判为误修复/精确匹配）以及汇总头部的模型标识（qwen2.5-coder:7b）均随 healreact/bench/cases/ 工件一并发布，作为本文结果的复现与审计依据。**代码与数据可用性**：上述脚本、JSON 工件、工件清单与论文快照已公开归档于 GitHub: https://github.com/vgawen/healreact；表 1–5 中的每一项原始计数（样本数、放弃/误修复/未解析、精确匹配）均可由该归档直接复算。工件清单汇总各脚本的运行命令、输入数据校验和、koenig 与 payload 的版本提交标识及模型版本。

3.2 共享 L3 协议

在 §3.3 与 §3.4 中，L3 修复器都经由 Ollama 以 temperature=0 调用 qwen2.5-coder:7b。用户提示包含：(i) 原样给出的失效 Playwright 选择器；(ii) 失效行附近的 ±3 行测试源码；(iii) 来自 LocatorSheet 的前 k 个候选，按加性词元重叠打分检索（testId ×3、dataAttr key/value ×2、aria/id/class/text/path ×1）。模型必须给出三行：SELECTOR、CANDIDATE_IDX、RATIONALE（纯文本，非 JSON：由于模型多次在选择器中输出未转义引号，本文未采用 JSON 输出）。解析器的多匹配冲突处理规则如下：按 (componentFile, line) 顺序的第一个匹配记录作为解析目标；多匹配的选择器会被标记，但指标计算中使用第一个命中。

表 1　误修复探针（§3.3）。两个提示变体产出完全一致的数字：软提示放弃约束并不降低隐性误修复。

提示变体	放弃 (好)	误修复 (坏)	未解析	**误修复 %**
VANILLA	13	59	3	**78.7%**
+ 放弃约束	13	59	3	**78.7%**

$n = 75$ 个可达 koenig 失效用例。真值目标已从候选集中移除；任何被给出的可解析选择器即为一次隐性误修复。78.7%（59/75）的 Wilson 95% 置信区间约为 [68.1%, 86.4%]。

3.3 目标缺失下的隐性误修复

3.3.1 设置

对 75 个可达的 koenig 失效用例（即失效版本提交处 LocatorSheet 中存在真值元素的用例），本文以相同的提示与检索条件重跑 L3 修复器，但将真值元素从 LocatorSheet 与候选集中强制移除，使模型在字面意义上无法选中正确答案。此时正确响应应当是放弃（CANDIDATE_IDX: -1）；任何给出可解析选择器的响应都是一次会沉默地将测试改写到错误元素上的隐性误修复。

文章比较两个提示变体：

1. VANILLA：与 L3 基线相同的系统提示。
2. ABSTAIN：同一提示外加一条明确的“如无候选契合则放弃”约束，告诉模型“一次沉默地把测试改写到错误元素上的盲修复永远比放弃更糟”。

3.3.2 结果

表 1 给出结果。

为排除“聚合计数偶然相同”的可能，表 2 给出两提示变体的逐用例配对交叉表。

3.3.3 解读

78.7% 是隐性误修复的上界，因为每个用例都是对抗性构造。在混合工作负载下，只有 L1 sheet 缺失目标的用例（本实验中是 18/93 = 19% 的 koenig 失效）面临此风险；朴素地相乘得到 $\approx 0.19 \times 0.79 = 15\%$，作为一个粗略的风险估计、而非测量到的部署率（对抗性探针的行为不必一对一地迁移到真正只在运行时可达的用例上）。即便作为风险估计，文章

表 2　Vanilla 与 Abstain 的逐用例配对交叉表（§3.3，$n = 75$）。所有非对角元素均为 0，表明两变体在每一个用例上给出完全相同的判定，而非仅聚合计数相同。

Vanilla \ Abstain	放弃	误修复	未解析
放弃 (好)	13	0	0
误修复 (坏)	0	59	0
未解析	0	0	3

在 `temperature=0` 的确定性解码下，75 个用例的逐例标签两两一致（非对角项 $= 0$，$\Delta = 0$）。

仍认为该风险已经高到不能允许 CI 无人值守地自动提交 LLM 提议的选择器补丁。

VANILLA 与 ABSTAIN 之间的等同性是更具诊断意义的发现，且这一等同性是逐用例成立的（表 2：75 个用例的配对判定无一发生改变）：一个 7B 规模的代码模型在该任务上不会因软提示式的放弃约束而改变其可观察行为。任何稳健的放弃机制都必须置于模型之外：例如确定性的检索分数阈值、锚点类别归属规则，或能够证伪通过补丁的行为验证器。该结果从实证层面支持将 HealReact 的 L4 层（行为重放验证器）作为下一层加以构建，而非依赖提示级别的安全机制。

3.4 L3 基线与两个低开销确定性手段

3.4.1 设置

在同样的 75 个可达 koenig 失效用例上，文章评估两个 L3 变体：

- v0（基线，两杠杆均关）：候选按与失效选择器的词元重叠程度检索，所有锚点（含 testId）统一权重；无后置过滤；系统提示要求模型优先选择 testId 类锚点。
- v1（两杠杆均开）：(i) 检索打分器把 testId 匹配加权（×3）排在 dataAttr key/value（×2）和 aria/id/class/text/path（×1）之上；(ii) 一个确定性强锚点后置过滤器：当所选候选携带可静态求值的 testId 但模型给出的不是 testId 选择器时，将其确定性地重写为 `getByTestId(testId)`；若 testId 来自不可求值的 JSX 表达式，则不做重写。系统提示不变。

为分离两个杠杆各自的贡献，文章进一步以二因子析因（testId 加权 $\in \{$关, 开$\}$ × 后置过滤 $\in \{$关, 开$\}$）跑满 4 个臂（表 4）。所有臂均使用 `qwen2.5-coder:7b`、`temperature=0`、同一系统提示；真值是 ReproBreak 中 `new_locator` 字符串所匹配的目标元素（经解析器解析）。

3.4.2 结果

表 3 汇总 L3 v0 与 v1 的端到端对比，表 4 给出 testId 加权检索与强锚点后置过滤的二因子析因结果。

表 3　L3 v0 → v1 端到端对比（75 个可达 koenig 失效）。v1 = v0 + testId 加权检索 + 强锚点后置过滤；v0 为两杠杆均关的统一权重基线。

指标	v0（两杠杆关）	v1（两杠杆开）
首位候选精确元素匹配	55 / 75 (73.3%)	**58 / 75 (77.3%)**
有效 Playwright 选择器	68 / 75 (90.7%)	68 / 75 (90.7%)
静态修复代理指标（全量 93）	55 / 93 (59.1%)	**58 / 93 (62.4%)**
强锚点后置过滤触发	0	1 / 75（正确）
新增正确用例（vs v0）	—	3
回归用例（vs v0）	—	0

表 4　两杠杆二因子析因（首位候选精确元素匹配，$n = 75$）。行 =testId 加权，列 = 强锚点后置过滤。每个杠杆单独带来 +2、两者合计 +3、零回归；方向一致但在该样本量下未达统计显著。

	过滤：关	过滤：开
加权：关	55 / 75	57 / 75
加权：开	57 / 75	**58 / 75**

McNemar 精确检验（双侧）：单开加权或单开过滤相对“两关”均为 $+2/-0$（$p = 0.50$）；两者全开相对“两关”为 $+3/-0$（$p = 0.25$）。所有差异方向一致、零回归，但在 $n = 75$ 下均未达 $p < 0.05$。

3.4.3 解读

两项手段均为低开销改动（一次正则优先级调整、一段约 10 行的后置过滤）、完全确定性、无需任何模型再训练。借助表 4 的二因子析因可分离两者贡献：以“两杠杆均关”为基线（55/75），单开 testId 加权检索与单开强锚点过滤各带来 +2 个精确匹配，两者全开合计 +3、且零回归；强锚点后置过滤器在过滤单开臂触发 2 次且均正确，在 v1 全开臂触发 1 次且正确，消除了部分“LLM 选中正确候选但在生成选择器时挂到了不稳定锚点上”的失败模式。须如实说明：这些增益方向一致、无一造成回归，但在 $n = 75$ 的样本量下经 McNemar 精确检验均未达统计显著（$p \geq 0.25$）；因此，文章把它们定位为“低开销、零回归的确定性工程杠杆”，而非已被统计确立的性能提升。该结果与 §3.3 相互呼应：在确定性介入有效之处，提示级介入的效果较为有限。

3.5 跨应用泛化与 BEM 盲区

3.5.1 设置

任何仅在单一应用上完成的实验都可能被审稿人合理质疑：所学到的可能只是应用专有的约

表 5　跨应用 L1 可达率（§3.5）。原始下降是真实的，但成因可诊断。

指标	koenig	payload
抽取器记录数（均值/单次）	每提交约 400	457（HEAD）
Playwright 失效配对	93（已复现）	319（CSV）
`new_locator` 可达	75 / 93 = **80.6%**	12 / 319 = **3.8%**
主导锚点	`data-kg-*`（约 50%）	CSS class†
模板字面量 class?	罕见	普遍（BEM）

†payload 的主导锚点占比来自对 307 个未命中用例的采样，而非全量失效分布。

定。为此本文以 `git sparse-checkout` 方式仅检出 `payloadcms/payload` [14] 的最新提交版本（HEAD；ReproBreak 中第二大的 Playwright 案例组，包含 319 个失效配对），在 `packages/ui/src` 上运行未修改的 L1 抽取器，产出 457 条记录。对每个失效配对，使用 `new_locator` 字符串在该记录表上做解析匹配（以最新提交版本可达率作为代理指标，逐版本提交配对未纳入本文）。

3.5.2 结果

原始 3.8% 的可达率较低，但对 307 个未命中用例的检查结果呈现出清晰的规律。payload 使用 BEM [15]：每个组件声明一个 `baseClass` 常量（例如 `const baseClass = 'dashboard'`），并以 `{`${baseClass}__add-button`}` 这样的模板字面量写出 className。测试引用*渲染后*的类 `.dashboard__add-button`，这对一个纯字面量的 AST 抽取器是不可见的。

3.5.3 解读

这一结果并非对 HealReact L1 设计的否证，而是识别出一处范围明确的盲区，其修复对应一次确定性的 AST 处理：构造一个 `baseClass` 解析器，回溯到常量声明并展开其模板字面量，并配以一个 CSS Modules [18] 等价的展开器。基于对 307 个未命中用例的*非系统采样*，本文*假设*修复后原始可达率会显著抬升，但该结论尚属假设而非测量；对全部 307 个未命中用例做系统化的失因标注（BEM、CSS Modules、运行时 DOM、解析器局限等类别计数）、固定并记录 payload 的最新提交版本 SHA 以及与 ReproBreak 的逐版本提交配对对齐，均留作后续工作（当前冻结工件仅记录了 payload 的仓库、配对数与记录数，未记录精确 HEAD SHA，属已知复现缺口）。3.8% 是当前抽取器下诚实的原始测量值。该发现真正的贡献是这条可证伪的跨应用泛化*假设*本身：锚点约定的多样性（`data-testid` vs 自定义 `data-*` vs BEM 模板字面量）很可能是定位器修复工具泛化能力的关卡，任何仅在单一应用上完成基准测试的工具都可能高估其泛化性。

4 讨论与局限

4.1 78.7% 对无人值守 CI 自愈意味着什么

LLM 自愈最常见的部署模式是高度自主化的：一个失败的端到端测试触发 LLM 给出补丁，CI 自动提交补丁或将其作为拉取请求（Pull Request，PR）评论发出。由 §3.3 的对抗性上界粗略换算，在不带行为验证器的此类部署中，约 15%（粗略上界估计，非实测部署率）的真实 Playwright 失效存在被朴素修复器静默改写到错误元素的风险。该风险的危险并不在于“修复器让构建失败”——构建失败本就是测试应承担的职责——而在于“修复器使构建通过，但通过对的是错误的目标”。

4.2 为什么软提示安全网会失效

Vanilla 与 Abstain 两种提示之间的等同性（表 1）并不令人意外。一种可能的解释是：系统提示中“必要时放弃”的软指令需与“生成合理输出”这一更强先验竞争，多数情况下处于劣势（此为机理层面的推测，本文未做训练动力学层面的验证）。对工具设计的启示是清晰的：反误修复的安全保障必须置于模型之外，例如确定性的检索分数阈值、锚点类别归属规则，以及一个能够证伪通过补丁的行为验证器。

4.3 为什么两个低开销确定性杠杆胜过提示工程

§3.4 的 testId 加权检索与强锚点后置过滤在机制上显而易见。经二因子析因分离，二者各带来 +2、合计 +3 个精确匹配且零回归（表 4）；增益方向一致，但在 $n = 75$ 下经 McNemar 精确检验未达统计显著。因此，文章既不声称其统计显著，也不声称该增益能泛化到当前设置以外，而是将其作为 HealReact 的一条局部设计经验：把锚点优先级约束放到候选池里和输出后置过滤里，比依靠系统提示让模型把约束内化更可靠，且不引入回归。

4.4 局限

*模型单一性带来的有效性威胁。*本文全部数字基于单一开源小型代码模型（`qwen2.5-coder:7b`）。前沿大模型（如 GPT-4 类）是否能在隐性误修复维度上显著缩小该差距尚待实测；根据 Vanilla = Abstain 等同性所揭示的机理，文章倾向于认为单凭规模不足以在缺乏外部确定性验证器的前提下闭合该差距。*基准数据集范围。*主结果来自单一 React+Playwright 代码库 koenig；§3.5 中已扩展至 payload，但其可达率受到 BEM 类名盲区的限制。*静态修复代理指标与运行时通过率。*77.3% / 62.4% 这两个 L3 数字反映的是选择器解析器层面的精确一致

表 6　解析器 strict/permissive 敏感性（koenig，$n = 93$）。permissive 为本文所采用的假设；strict 关闭“动态属性值视为可能匹配”与“同文件祖先近似”两处过近似。

指标	permissive（本文）	strict（下界）
new_locator 可达	75 / 93 (80.6%)	51 / 93 (54.8%)
多匹配（≥ 2 命中）	46	0
未解析（0 命中）	18	42

性。其与真实 Playwright 运行时通过率之间的换算关系，需要在 ReproBreak 提供的逐版本提交复现环境中通过 Docker 重放修复后的测试加以建立。作为最小可行性检查，文章对 3 个 v1 静态精确匹配用例（编号 615、702、703）做了 Docker 运行时重放：三者的修复基线均通过；替换为 HealReact 提议选择器后，2/3 运行通过，1/3 运行失败（编号 702，等待 `getByTestId('color-picker-toggle')` 超时）。这一小规模验证实验不能估计全量通过率，但说明静态精确匹配既有运行时可迁移的案例，也会因可见性/时序/上下文差异而失败；因此本文继续把 77.3% / 62.4% 标为静态修复代理指标，而不宣称端到端通过率。

4.5 有效性威胁

78.7% 这一数字依赖解析器语义。`resolve_locators.py` 把动态 JS 表达式属性值视为可能匹配、对复合 CSS 使用祖先过近似；这两项决策在错误排查场景下合理，但在边界情况下偏松。文章在解析器假设审计中撤回了早期“可达率上限 98%”的虚抬声明，在收紧解析器假设后以 80.6% 作为保守数字。由于解析器是本文所有“正确匹配/误修复/可达率”判定的事实判定器，其判定边界会直接影响这些数字，文章做了一次 strict/permissive 双假设的敏感性扫描（表 6）。本文主要实验结果中的 80.6%（75/93）来自 permissive 假设；若关闭上述两处过近似改用 strict 假设，可达率降至 54.8%（51/93）。因此 L1 静态可达率的真实区间应理解为 [54.8%, 80.6%]，80.6% 是上界。此外，permissive 下 75 个可达用例中有 46 个为多匹配（解析器按 `(componentFile, line)` 顺序取首个命中消歧），未解析用例从 permissive 的 18 个增至 strict 的 42 个。对抽样用例用真实 DOM 快照交叉校验解析判定，仍留作后续工作。

4.6 未来工作

本文的诊断结论自然延伸出两类可推进的工程化方向：其一，将静态基底向跨锚点约定（BEM 模板字面量、CSS Modules 等）扩展，使可达率在不同 React 项目中保持稳定；其二，沿 §3 给出的经验性必要性，构建并集成一个基于网络行为重放的轻量级验证器，作为 LLM 修复结果的下游安全机制。

5 结论

在缺乏行为验证器的部署条件下，LLM 驱动的 Playwright 自愈与“测试通过但真实回归被遮蔽”之间仅一步之遥。本文在对抗性极限下、于选择器解析层面测量到 78.7% 的隐性误修复上界，而提示级放弃指令所带来的缓解恰好为零；该数字为静态诊断指标，与运行时测试通过率之间的换算仍需基于 Docker 的真实重放。HealReact 的 L1 基底使 koenig 数据集中 80.6% 的真实失效在静态层面可达；一个小型本地修复器在可达用例上首位候选选择器的精确匹配率达到 77.3%；两个低开销确定性手段进一步给出了帕累托改进。三项结果合起来，为后续工作中评估行为重放验证器给出了直接的经验性动机，也给出了一条由锚点约定差异所限制的、可证伪的跨应用泛化假设。

参考文献

[1] 李聪, 蒋炎岩, 许畅. 基于 GUI 事件的安卓应用录制重放关键技术综述[J]. 软件学报, 2022, 33 (5): 1612-1634.

[2] 王博, 陈冲, 邓明, 等. 移动应用 GUI 测试自动生成技术综述[J]. 软件学报, 2025, 36(6): 2713-2746.

[3] 杨丽波. 软件自动化测试方法的发展现状分析[J]. 无线互联科技, 2017(4): 49-50.

[4] 刘宝银. 软件测试技术的应用及其发展形势分析[J]. 无线互联科技, 2022, 19(12): 38-40.

[5] Healenium Contributors. Healenium: self-healing library for Selenium-based tests[EB/OL]. 2024[2026-06-20]. https://healenium.io.

[6] JOSEPH R N. Beyond LLM-based test automation: a zero-cost self-healing approach using DOM accessibility tree extraction[EB/OL]. 2026[2026-06-20]. https://arxiv.org/abs/2603.20358.

[7] DE MOURA T S, ADAMIETZ L, MEHBOOB S, et al. ReproBreak: a dataset of reproducible web locator breaks[EB/OL]. 2026[2026-06-20]. https://arxiv.org/abs/2605.12158.

[8] 香佳宏, 徐霄阳, 孔繁初, 等. 大模型在软件缺陷检测与修复的应用发展综述[J]. 软件学报, 2025, 36(4): 1489-1529.

[9] 李岩, 杨文章, 张翼, 等. 基于大语言模型的模糊测试研究综述[J]. 软件学报, 2025, 36(6): 2404-2431.

[10] HUI B, YANG J, CUI Z, et al. Qwen2.5-Coder technical report[EB/OL]. 2024[2026-06-20]. https://arxiv.org/abs/2409.12186.

[11] Ghost Foundation. koenig-lexical: Ghost's Lexical-based rich-text editor[EB/OL]. 2025[2026-06-20]. https://github.com/tryghost/koenig.

[12] 李斌, 贺也平, 马恒太. 程序自动修复: 关键问题及技术[J]. 软件学报, 2019, 30(2): 244-265.

[13] Meta Open Source. Lexical: an extensible text editor framework[EB/OL]. 2025[2026-06-20]. https://lexical.dev.

[14] Payload CMS Inc. Payload CMS: headless Node.js CMS on Next.js[EB/OL]. 2025[2026-06-20]. https://payloadcms.com.

[15] Yandex. BEM: block, element, modifier CSS methodology[EB/OL]. 2009[2026-06-20]. https://en.bem.info/methodology/.

[16] SHERRET D. ts-morph: a TypeScript compiler API wrapper for programmatic code manipulation[EB/OL]. 2024[2026-06-20]. https://github.com/dsherret/ts-morph.

[17] Ollama Contributors. Ollama: local inference for open-weights LLMs[EB/OL]. 2024[2026-06-20]. https://ollama.com.

[18] CSS Modules Authors. CSS Modules: locally-scoped CSS class names[EB/OL]. 2020[2026-06-20]. https://github.com/css-modules/css-modules.